

Projet Cours IA

L3 INFO International

2026

Prof Nazim Agoulmine

Jeu de Pacmen

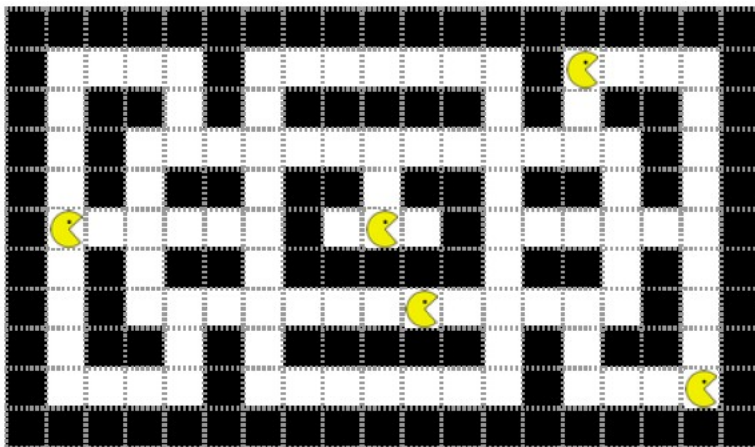
Considérons le problème du contrôle simultané de $n=5$ pacmen.

Plusieurs pacmen peuvent se trouver sur la même case au même moment, et à chaque unité de temps, chaque pacman se déplace d'au plus une case verticalement ou horizontalement (en d'autres termes, un pacman peut s'arrêter, et plusieurs pacmen peuvent aussi se déplacer simultanément). Le but du jeu est de faire en sorte que tous les pacmen arrivent sur la même case en un minimum d'unités de temps.

Dans ce travail, utilisez la notation suivante :

Soit M le nombre de cases dans le labyrinthe qui ne sont pas des murs (c'est-à-dire le nombre de cases où les pacmen peuvent aller) ; n le nombre de pacmen et $p_i = (x_i, y_i) : i = 1 \dots n$ la position du pacman i .

Supposez que le labyrinthe est connexe (c'est-à-dire que toutes les cases sont accessibles par les pacmen).



1. Formulez le problème.
2. Quelle est la **taille exacte de l'espace d'états** ?
3. Donner la limite supérieure du **facteur de branchement** de ce problème.

4. Donner la limite du nombre de nœuds développés par l'exploration en arbre à coût uniforme (UCS). **Justifiez votre réponse.**

5. Est-ce que les heuristiques suivantes sont **admissibles** ? **Justifiez.**

- a) Le nombre de paires (ordonnées) (x, y) de pacmen avec des coordonnées différentes.
- b) $h((x_1, y_1), \dots, (x_n, y_n)) = 1/2 \max \{ \max_{i,j} |x_i - x_j|, \max_{i,j} |y_i - y_j| \}$
(La moitié de la valeur maximale entre la plus grande différence des x_i et celle des y_i)

6. Proposer deux **algorithmes** pour solutionner le problème en utilisant l'**algorithme DBF** de manière récursive puis avec l'**algorithme A*** en utilisant une pile.

7. Programmer les deux solutions en utilisant le langage Python.

- Vous devez modéliser le labyrinthe à l'aide d'une structure grid simple (0 pour case vide et 1 pour case qui représente un mur). Par exemple :

```
grid = [  
    [0, 0, 0, 1, 0],  
    [0, 1, 0, 1, 0],  
    [0, 0, 0, 0, 0],  
    [0, 1, 1, 1, 0],  
    [0, 0, 0, 0, 0]  
]
```

- Vous devez utiliser la classe **Etat** pour représenter l'état du système avec les positions des pacmen, le coût actuel (**g**), l'heuristique (**h**), et le coût total (**f**).
- Vous devez implémenter les fonctions suivantes :
 - La fonction **heuristic** pour calculer la distance maximale entre tous les pacmen pour estimer le coût restant.
 - La fonction **gen_voisins** pour générer tous les états voisins possibles en déplaçant chaque pacman dans une des quatre directions.
 - La fonction **A_star** pour implémenter l'algorithme A*, en utilisant une file de priorité pour explorer les états avec le coût total le plus bas en premier. Python ne dispose pas d'une structure de données de file de priorité intégrée, mais il offre la bibliothèque **heapq**, qui peut être utilisée pour créer une file de priorité. **heapq** utilise une structure de tas (**heap**) pour maintenir les éléments dans l'ordre de priorité de manière efficace. Utiliser cette structure pour dans le cadre de l'algorithme A*, pour stocker tous les états (ou nœuds) qui doivent être explorés, triés par leur coût total ($g + h$) et pour pouvoir retirer en priorité l'élément avec le coût total le plus bas (c'est-à-dire l'état le plus prometteur à explorer, ce qui est essentiel pour l'algorithme A*).

8. Afficher et comparer les **temps d'exécution des deux solutions** pour des configurations de départ aléatoires de positionnement des pacmen. **Commenter les résultats.**